

Ejemplos

Raul Gomez

2025-12-06

Protocolo de Encriptación RSA

En este notebook, mostraremos el proceso de encriptación RSA. Empezaremos por cargar las funciones que necesitamos:

```
import sys

if 'google.colab' in sys.modules:
    print("Running on Colab. Installing package from GitHub...")
    !pip install git+https://github.com/Krul-dev/MA2006B.git
else:
    print("Running locally. Using local package installation.")

from cryptocalc import (
    rsa_key_generation,
    rsa_encryption,
    rsa_decryption,
    encode,
    decode
)
```

Running locally. Using local package installation.

Una vez que ya hemos cargado las librerías necesarias, empezamos por generar nuestras llaves *pública* y *privada*

```
(public_key, private_key) = rsa_key_generation(1000)
```

En este caso la llave privada consiste de dos partes. Es muy importante que esta información no se divulgue.

Etapa 1. Conociendo el negocio

```
private_key
```

```
2218313791177302398626113289620983184626874039510011561757459541853683012405342137407443154518
```

La llave pública, por otro lado, se debe de compartir con las partes interesadas, afín de que puedan encriptar sus mensajes. En este caso la llave pública a compartir es:

```
public_key
```

```
(539381327594387946157646061070835703920676642655280877535117563218640261893284964561456811724  
487717997731496097266546136728139325185961434887757831930017288351217166358739782614556521338
```

Para demostrar como funciona este proceso, procedemos ahora a generar un mensaje que deseamos encriptar. Empezaremos con el simple 'Hello World!'

```
plain_text = 'Hello World!'
```

Antes de poder encriptar este mensaje, necesitamos codificarlo en forma numérica:

```
plain_message = encode(plain_text)  
plain_message
```

```
10334410032597741434076685640
```

Utilizando la llave pública, podemos ahora generar el *mensaje encriptado*

```
encrypted_message = rsa_encryption(public_key, plain_message)  
encrypted_message
```

```
2699354751513600896433388925582272754964955468753117310425987457709000426272857423395249300388
```

Si este mensaje fuera interceptado por un adversario (digamos Eva) este sería el mensaje de texto que obtendría:

```
encrypted_text = decode(encrypted_message)  
encrypted_text
```

```
'Bá+ËCÅtÛâS;`x8e·$Ü£\x08ªßÉ\n0ó%Î\x15;\x81W\x933ª&6\x98Õ?\x9dÕö\x85¬A^\x838ü·\x87`CÉÅ«ß«â\x02
```

Por otro lado, Alicia al momento de recibir este mensaje podría desencriptarlo primero utilizando su llave pública:

Etapa 1. Conociendo el negocio

```
decrypted_message = rsa_decryption(public_key,private_key, encrypted_message)
decrypted_message
```

10334410032597741434076685640

Para poder leer este mensaje basta con decodificarlo a mensaje de texto:

```
decrypted_text = decode(decrypted_message)
decrypted_text
```

'Hello World!'